

Réduction de dimensions

ACP avec python

Fichier exemple: voitures.csv

```
X = pd.read_csv(« voitures.csv »)
```

```
#dimension print(X.shape) # (18, 6)
```

```
#nombre d'observations n = X.shape[0]
```

```
#nombre de variables p = X.shape[1]
```

```
#affichage des données print(X)
```

18 voitures et 6 variables numériques continues

On va centrer et réduire les données

```
#scikit-learn import sklearn
```

```
#vérification de la version print(sklearn.__version__)
```

Nousinstancions l'objet et nous l'appliquons sur la matrice X. Nous obtenons une matrice Z

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{\sigma_j}$$

Où $\bar{x}_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$ est la moyenne de la variable X_j , $\sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2}$ son écart-type.

On va centrer et réduire les données

```
import sklearn
```

```
#vérification de la version  
print(sklearn.__version__)
```

Nousinstancions l'objet et nous l'appliquons sur la matrice X. Nous obtenons une matrice Z

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{\sigma_j}$$

Où $\bar{x}_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$ est la moyenne de la variable X_j , $\sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2}$ son écart-type.

```
from sklearn.preprocessing import StandardScaler
```

```
sc = StandardScaler()
```

```
Z = sc.fit_transform(X)
```

```
print(Z)
```

On vérifie les ppts du nouveau dataset (Z)

```
Import numpy as np
```

```
print(np.mean(Z,axis=0))
```

```
print(np.std(Z,axis=0,ddof=0))
```

```
from sklearn.decomposition import PCA
```

```
acp = PCA(svd_solver='full')
```

```
#Affichage des paramètres  
print(acp)
```

```
#Calculs  
coord = acp.fit_transform(Z)
```

```
#On vérifie que le nombre de CP est le nombre de variables  
print(acp.n_components_)
```

```
#Calcul des valeurs propres
```

```
eigval = (n-1)/n*acp.explained_variance_
```

```
print(eigval)
```

```
#proportion de variance expliquée
```

```
print(acp.explained_variance_ratio_)
```

```
#scree plot
```

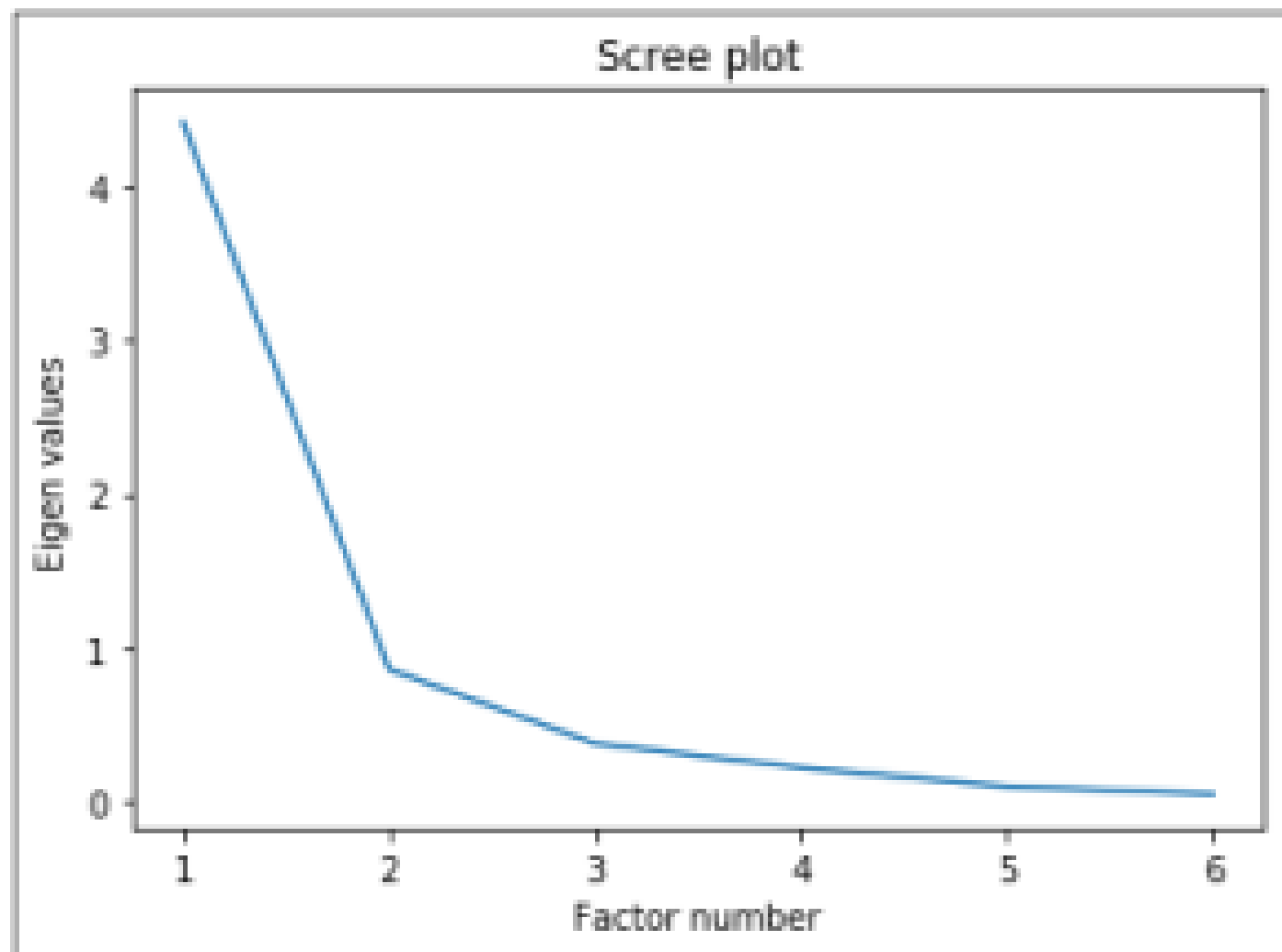
```
plt.plot(numpy.arange(1,p+1),eigval)
```

```
plt.title("Scree plot")
```

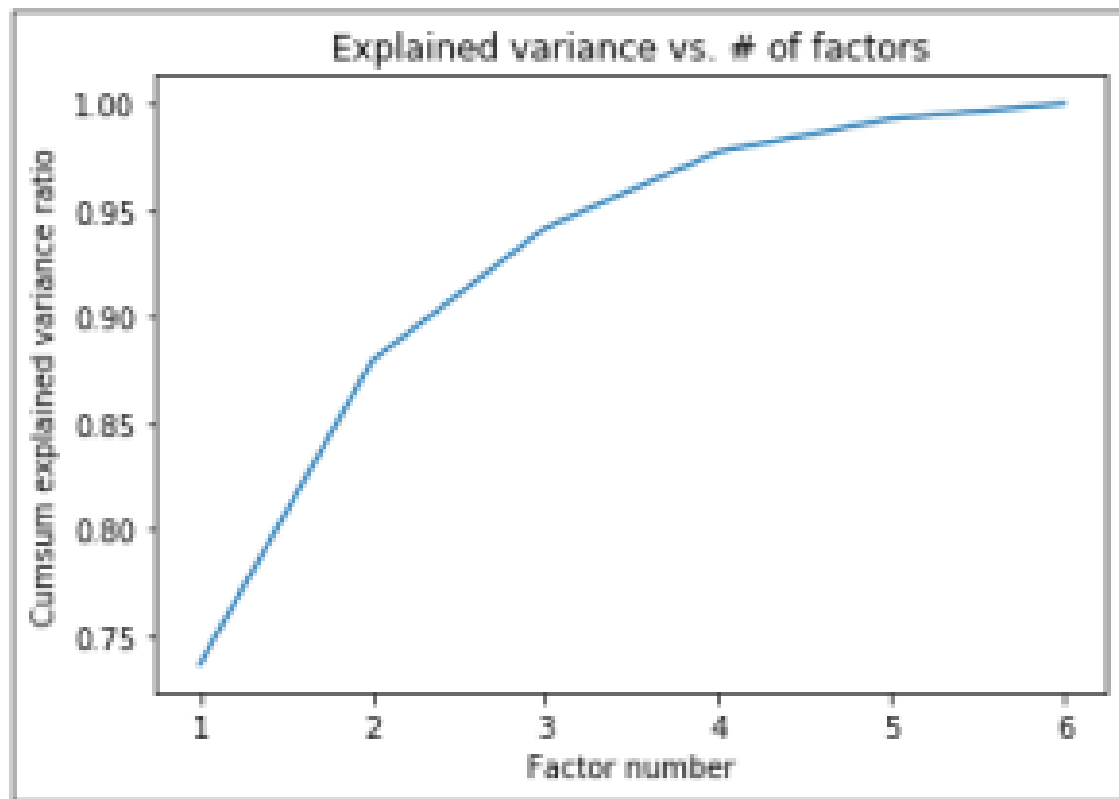
```
plt.ylabel("Eigen values")
```

```
plt.xlabel("Factor number")
```

```
plt.show()
```

```
plt.plot(numpy.arange(1,p+1),numpy.cumsum(acp.explained_variance_ratio_))  
plt.title("Explained variance vs. # of factors")  
plt.ylabel("Cumsum explained variance ratio")  
plt.xlabel("Factor number")  
plt.show()
```



Déterminer le nombre de facteurs à retenir

Les “cassures” dans les graphiques sont souvent évoquées (règle du coude) pour identifier le nombre de facteurs K^* à retenir.

La solution ($K^* = 2$) semble s'imposer ici.

Les seuils sont définis par :

$$b_k = \sum_{m=k}^p 1/m$$

Le facteur n°k est validé si ($\lambda_k > b_k$), où λ_k est la valeur propre associée à l'axe n°k

Représentation des individus

Les coordonnées factorielles (Fik) des individus ont été collectées dans la variable coord.

Nous les positionnons dans le premier plan factoriel avec leurs labels pour situer et comprendre les proximités entre les véhicules)

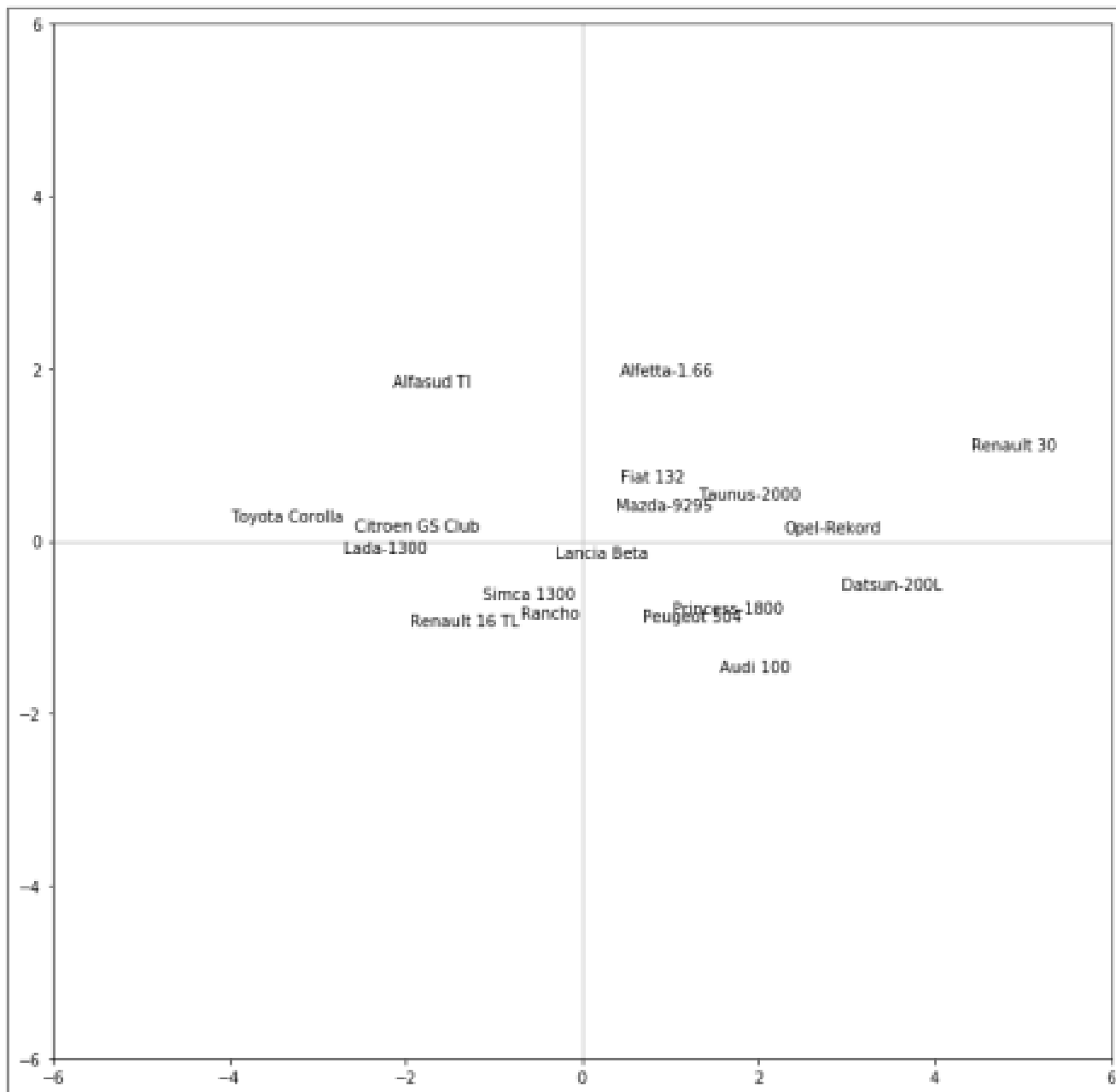
```
#positionnement des individus dans le premier plan
```

```
fig, axes = plt.subplots(figsize=(12,12))  
axes.set_xlim(-6,6) #même limites en abscisse  
axes.set_ylim(-6,6) #et en ordonnée
```

```
#placement des étiquettes des observations  
for i in range(n):  
    plt.annotate(X.index[i],(coord[i,0],coord[i,1]))
```

```
#ajouter les axes  
plt.plot([-6,6],[0,0],color='silver',linestyle='-',linewidth=1)  
plt.plot([0,0],[-6,6],color='silver',linestyle='-',linewidth=1)
```

```
#Affichage  
Plt.show()
```



Représentation des individus

Les coordonnées factorielles (Fik) des individus ont été collectées dans la variable coord.

Nous les positionnons dans le premier plan factoriel avec leurs labels pour situer et comprendre les proximités entre les véhicules)

Qualité de la représentation: les COS^2

Pour calculer la qualité de représentation des individus sur les axes, nous devons d'abord calculer les carrés des distances à l'origine des individus, qui correspondent également à leur contribution dans l'inertie totale

```
#contribution des individus dans l'inertie totale
```

```
di = numpy.sum(Z**2,axis=1)
```

```
print(pd.DataFrame({'ID':X.index,'d_i':di}))
```

```
#qualité de représentation des individus – les cos2
```

```
cos2 = coord**2
```

```
for j in range(p):
```

```
    cos2[:,j] = cos2[:,j]/di
```

```
print(pd.DataFrame({'id':X.index,'COS2_1':cos2[:,0],'COS2_2':cos2[:,1]}))
```

Représentation des individus

Les coordonnées factorielles (Fik) des individus ont été collectées dans la variable coord.

Nous les positionnons dans le premier plan factoriel avec leurs labels pour situer et comprendre les proximités entre les véhicules)

Contributions aux axes des voitures

```
ctr = coord**2
for j in range(p):
    ctr[:,j] = ctr[:,j]/(n*eigval[j])

print(pd.DataFrame({'id':X.index,'CTR_1':ctr[:,0],'CTR_2':ctr[:,1]}))
```

Représentation des variables

```
print(acp.components_)
```

```
#racindes carrée des valeurs propres
```

```
sqrt_eigval = numpy.sqrt(eigval)
```

```
#corrélation des variables avec les axes
```

```
corvar = numpy.zeros((p,p))
```

```
for k in range(p):
```

```
    corvar[:,k] = acp.components_[k,:] * sqrt_eigval[k]
```

```
#afficher la matrice des corrélations variables x facteurs
```

```
print(corvar)
```

Les variables sont en ligne et les facteurs en colonnes